

ASVA

Technical Requirements Document (TRD)

Version: 1.0 **Platform:** Responsive Web App / PWA

Primary Interfaces: Paramedic App and Hospital Dashboard

Core Integrations: Cloudinary, Snowflake, Google Authentication, Maps/Traffic

1. Technical Architecture

ASVA uses a frontend-backend architecture. The paramedic application captures emergency data, voice input, GPS and medical reports. The hospital dashboard receives authorized, hospital-specific emergency information in real time. The backend coordinates authentication, emergency state, hospital selection, routing, location updates, voice extraction, Cloudinary uploads, Snowflake storage, authorization and WebSocket events.

PARAMEDIC APP → ASVA BACKEND → Snowflake / Cloudinary / Maps & Traffic → HOSPITAL DASHBOARD
The ambulance GPS stream also feeds the hospital and traffic-control integration.

2. Recommended Technology Stack

Frontend

React + TypeScript + Vite. Tailwind CSS and shadcn/ui for the interface. WebSocket/Socket.IO for real-time updates. Google Maps Platform or another approved mapping provider for maps, directions, traffic-aware routing and ETA.

Backend

Node.js + TypeScript + Express.js. REST APIs for standard operations and WebSockets for live location, vitals, emergency and report events.

Data

Snowflake is the structured emergency data and analytics layer. Cloudinary stores actual medical documents and images. Google OAuth is used for hospital dashboard authentication.

3. Backend Responsibilities

Services

Emergency management; ambulance management; hospital discovery and selection; routing; GPS ingestion; voice processing; medical information extraction; report management; Snowflake persistence; Cloudinary integration; authentication and RBAC; WebSocket events; notifications; audit logging.

Folder Structure

backend/src/config | controllers | services | middleware | routes | websocket | utils | server.ts

4. Frontend Architecture

Structure

frontend/src/components | pages | services | hooks | store | types | App.tsx

State

Use a lightweight state manager such as Zustand for emergency, ambulance, hospital, route, vitals, reports, location and connection state.

5. Paramedic Application

Active Emergency

The registered ambulance is already known to ASVA. When a patient is received, the paramedic starts an emergency case and ASVA generates an Emergency Case ID. Patient name is optional and must not block the workflow.

Hospital Selection

Show suitable hospitals using distance, available beds and relevant emergency capability/availability where applicable. The paramedic selects the destination hospital.

Navigation

Show current location, destination, recommended route, traffic condition, distance and ETA. Recalculate when meaningful traffic changes occur.

Medical Monitoring

Capture only important emergency parameters: emergency type, severity, consciousness, key symptoms, relevant vitals and immediate treatment. Avoid long forms and unnecessary questions.

6. Voice-First Medical Data Entry

Goal

Voice input is not merely a transcript field. ASVA converts natural paramedic speech into structured medical fields.

Pipeline

Voice → Speech-to-Text → Medical Entity Extraction → Field Mapping → Validation → Structured JSON → Paramedic Confirmation → Snowflake.

Example

Speech such as "Patient is unconscious, heart rate 120, oxygen saturation 86, blood pressure 90 over 60" becomes structured values for consciousness, heart rate, SpO2 and blood pressure.

Safety

Ambiguous values must be flagged for confirmation. The system must not silently assume or invent clinical values. The paramedic can edit every extracted value.

7. Medical Report Management

Upload

Paramedics can upload ECGs, blood tests, X-rays, CT/MRI images, prescriptions, previous medical reports and other relevant files.

Cloudinary

Cloudinary stores the actual medical files and provides asset identifiers and controlled delivery/access. Recommended organization: ASVA/emergencies/{Emergency_ID}/reports/{REPORT_TYPE}/.

Snowflake

Snowflake stores report metadata such as Report ID, Emergency ID, Ambulance ID, Report Type, timestamp, uploader and Cloudinary asset reference.

Viewer

Hospital dashboard displays report metadata. Clicking View Report calls the backend, which authenticates the hospital user, verifies access to the emergency, retrieves the Cloudinary reference and returns controlled document access.

8. Snowflake Data Architecture

Tables

AMBULANCES: ambulance_id, vehicle_number, status, current coordinates, registered_at.

HOSPITALS: hospital_id, name, coordinates, available_beds, emergency_status, capabilities, status.

EMERGENCY_CASES: emergency_id, ambulance_id, emergency_type, severity, hospital_id, status, created_at, updated_at, arrived_at.

VITALS: vital_id, emergency_id, heart_rate, blood_pressure, spo2, respiratory_rate, temperature, consciousness, recorded_at.

AMBULANCE_LOCATIONS: location_id, ambulance_id, emergency_id, latitude, longitude, eta, recorded_at.

MEDICAL_REPORTS: report_id, emergency_id, ambulance_id, report_type, cloudinary_public_id, cloudinary_url/reference, uploaded_by, uploaded_at.

TREATMENTS: treatment_id, emergency_id, treatment_type, description, recorded_by, recorded_at.

HOSPITAL_USERS: user_id, Google ID, email, hospital_id, role, created_at.

Principle

Do not store large medical binaries in Snowflake in this architecture. Snowflake stores structured information and references to Cloudinary assets; Cloudinary stores the actual documents and images.

9. Hospital Authentication and Authorization

Authentication

Hospital dashboard uses Google Authentication. A successful Google login must be mapped to an authorized hospital account before access is granted.

Rbac

Roles can include PARAMEDIC, HOSPITAL, TRAFFIC_CONTROL and ADMIN. Hospital requests must be scoped server-side to the authenticated hospital_id.

Privacy

A hospital sees only ambulances that have selected that hospital as their destination. Traffic control receives only the information required for traffic coordination, not medical documents or unnecessary patient information.

10. Hospital Dashboard

Main View

The primary screen is a live map showing only ambulances coming to the authenticated hospital. A prioritized incoming list appears alongside the map.

Prioritization

Sort primarily by severity/criticality, then by distance or ETA within the same severity. Example ordering: Critical → High → Moderate → Low; ties are resolved by nearest ETA.

Case Detail

Clicking an ambulance opens Emergency ID, emergency type, severity, ETA, live location, current vitals, condition, treatment and medical reports. Reports can be opened through the controlled Cloudinary access flow.

Realtime

Dashboard receives ambulance.location.updated, vitals.updated, emergency.updated, report.uploaded, severity.changed, eta.updated and ambulance.arrived events.

11. Live Location and Traffic Control

Gps

The ambulance sends latitude, longitude, timestamp and ETA through WebSocket or a location API. The backend distributes updates to the destination hospital and traffic-control integration.

Traffic Payload

Ambulance ID, Emergency ID, coordinates, destination hospital, ETA, severity and timestamp. Avoid sending medical reports or unrelated medical data.

Arrival

Use hospital geofencing to detect arrival. Transition the emergency from EN_ROUTE/APPROACHING to ARRIVED and remove it from the hospital's active incoming queue.

12. API Specification

Auth

POST /api/auth/google
GET /api/auth/me
POST /api/auth/logout

Emergency

POST /api/emergencies
GET /api/emergencies/:id
PATCH /api/emergencies/:id
POST /api/emergencies/:id/complete

Hospital

GET /api/hospitals/recommended
GET /api/hospitals/:id
POST /api/emergencies/:id/select-hospital

Location

POST /api/location
GET /api/emergencies/:id/location
WebSocket preferred for continuous updates.

Vitals

POST /api/emergencies/:id/vitals
GET /api/emergencies/:id/vitals

Voice

POST /api/voice/analyze. Response contains extracted fields and confidence values for paramedic confirmation.

Reports

POST /api/reports/upload-signature

POST /api/reports

GET /api/emergencies/:id/reports

GET /api/reports/:id/access

Hospital

GET /api/hospital/incoming

GET /api/hospital/emergency/:id

13. Real-Time WebSocket Events

Events

ambulance.location.updated

vitals.updated

emergency.updated

report.uploaded

severity.changed

eta.updated

ambulance.arrived

Channel

After authentication, a hospital subscribes to a hospital-specific channel such as hospital:{hospitalId}. The backend must ensure that only authorized events for that hospital are delivered.

14. Emergency State Machine

States

CREATED → PATIENT_RECEIVED → HOSPITAL_SELECTED → EN_ROUTE → APPROACHING → ARRIVED → COMPLETED. Cancellation can occur from an active state where appropriate.

15. Security Requirements

Requirements

Google OAuth for hospital users; server-side RBAC; hospital-level data isolation; controlled Cloudinary document access; audit logging for sensitive report access; secrets stored only in server-side environment variables; no public access to medical reports.

16. Performance Targets

Targets

Initial dashboard load: target under 2 seconds under normal conditions. Location updates: approximately every 1–5 seconds depending on network and device constraints. Hospital dashboard propagation: target under 2 seconds after backend receipt under normal conditions. Short voice processing: target under 5 seconds. Normal report upload: target under 5 seconds subject to network and file size.

17. Error Handling

Gps

If GPS is unavailable, show an explicit location-unavailable state and retry. Do not present stale coordinates as current.

Voice

If extraction is uncertain, highlight the field and require confirmation.

Upload

If Cloudinary upload fails, provide retry without blocking the emergency workflow.

Data

For production, use a durable queue/cache for transient data-service failures. For the hackathon, implement clear retry and failure states.

18. Analytics

Snowflake Analytics

Use accumulated structured data to calculate average response time, transport time, hospital preparation time, emergency distribution, critical-case volume, hospital performance and ambulance utilization. Analytics should be derived from structured Snowflake data.

19. Deployment

Recommended

Frontend: Vercel or equivalent. Backend: Render/Railway or equivalent. Snowflake: managed Snowflake account. Cloudinary: managed Cloudinary account. Maps/traffic: approved maps provider. Use HTTPS in deployed environments.

20. Hackathon MVP Priority

Must Have

1. Registered ambulance and Emergency ID. 2. Hospital recommendation using distance and beds. 3. Hospital selection. 4. Route, traffic and ETA. 5. Live location. 6. Voice-to-structured medical fields. 7. Critical vitals. 8. Medical report upload to Cloudinary. 9. Report metadata in Snowflake. 10. Google-authenticated hospital dashboard. 11. Hospital-specific live map. 12. Severity + ETA prioritization. 13. Metadata → Cloudinary report viewer.

21. Final Technical Flow

PATIENT RECEIVED → CREATE EMERGENCY → FIND HOSPITAL → SELECT HOSPITAL → ROUTE + TRAFFIC → LIVE LOCATION → CRITICAL VITALS → VOICE TO STRUCTURED DATA → MEDICAL REPORTS → SNOWFLAKE + CLOUDINARY → HOSPITAL DASHBOARD → PRIORITIZED INCOMING AMBULANCES → ER PREPARATION → PATIENT ARRIVAL.

Technology Roles

Cloudinary = actual medical documents and images. Snowflake = structured emergency data, report metadata, operational data and analytics. ASVA backend = orchestration, access control, business logic and real-time delivery. Voice AI = natural paramedic speech to structured medical information. Maps/Traffic = routing, ETA and ambulance movement. Google Authentication = restricted hospital dashboard access.